Rectilinear Polygon Decomposition into Convex Pieces and Rectangles

Wai-Shing Luk

***Abstract*—We study the decomposition of rectilinear polygons into convex pieces and rectangles using a compressed even-index representation. Three recursive cut policies are considered: convex, explicit, and implicit. We show that the natural composition of an implicit cut followed by an explicit cut is not a rectangle decomposition. The implicit policy only tests corners that are implicit in the representation, so a reflex corner that happens to be a stored vertex is invisible to it; the explicit policy cannot rectangularize a piece that is already orthogonally convex. We propose a corrected two-stage algorithm that first partitions the polygon into orthogonally convex pieces and then slices each piece into rectangles with vertical slabs. We prove that the slab stage yields a partition into rectangles for x-monotone pieces, analyze the resulting $O(n^2)$ time complexity, and report a C++ implementation verified on a twelve-point fixture (seven rectangles, total area 20) and a 138-case test suite. The same fixture provides a concrete counterexample showing that area preservation alone does not certify a rectangle decomposition.**

I. Introduction

Rectilinear polygons are ubiquitous in the physical design of integrated circuits. Wires, cell boundaries, blockages, and many layout rules are Manhattan: every edge is horizontal or vertical. A layout block is often given as a rectilinear polygon, while downstream tools prefer a collection of rectangles because rectangles are easy to store, query, and check against design rules. Decomposing a rectilinear polygon into convex pieces, and then into rectangles, is therefore a basic geometric preprocessing step in floorplanning, routing-region generation, mask fracture, and area or timing estimation.

Two decomposition problems are often conflated. A *convex decomposition* partitions a rectilinear polygon into orthogonally convex pieces, i.e., pieces that are simultaneously x-monotone and y-monotone. A *rectangle decomposition* partitions it into rectangles. Orthogonal convexity is weaker than rectangularity: an L-shape is orthogonally convex but is not a rectangle. A convex decomposition is a useful intermediate result, but it does not by itself solve the rectangle decomposition problem.

This paper revisits a family of recursive cut algorithms that operate on a compressed representation of a rectilinear polygon: only every other vertex is stored, and the omitted vertices are reconstructed on demand. The representation halves the storage and leads to simple base cases: two stored points describe a rectangle, three describe an L-shaped monotone piece, and more points describe a larger orthogonally convex polygon. Three cut policies are studied. The *convex* policy cuts until the pieces are orthogonally convex. The *explicit* policy uses the same recursion but omits the three-point shortcut. The *implicit* policy cuts at corners that are implicit in the representation.

The main observation of this paper is that the composition "implicit cut, then explicit cut on each piece" is not a rectangle decomposition, even though it preserves area and returns convex pieces. The implicit policy can only find reflex corners that are implicit vertices; a reflex corner that is stored is invisible to its test. The explicit policy, in turn, stops on a piece that is already orthogonally convex, so it cannot split the residual piece into rectangles. On a twelve-point fixture, the composition returns six pieces, including a zero-width piece and a five-point piece of area 14 that is not a rectangle. The area sum is correct, which is precisely why the defect can escape area-based tests.

We propose a corrected pipeline. The polygon is first decomposed into orthogonally convex pieces by the convex policy. Each convex piece is then rectangularized by vertical slabs: the piece is sliced between consecutive distinct x-coordinates of its vertices, and each slab contributes one rectangle because an x-monotone polygon meets a vertical line in a single interval. The result is a partition into rectangles, stored in the same compressed two-point form as the rest of the library. We prove the correctness of the slab stage for x-monotone pieces, analyze the $O(n^2)$ complexity, and report a C++ implementation with a test suite that asserts the output class rather than only the area.

The contributions are:

1. a precise description of the compressed even-index representation and of the shared recursive cut skeleton;
2. exact statements of the convex, explicit, and implicit cut predicates, including the orientation-dependent sign test and the degenerate-cut handling;
3. a counterexample proving that the implicit policy has a blind spot at stored reflex corners, and that implicit-then-explicit is not a rectangle decomposition;
4. a corrected convex-plus-slab rectangle algorithm with a correctness proof for x-monotone pieces and an $O(n^2)$ complexity bound;
5. an implementation and verification on a twelve-point fixture and a 138-case test suite, including the seven rectangles and their areas.

The remainder of the paper is organized as follows. Section

2 introduces the representation. Section 3 presents the recursive cut framework. Sections 4, 5, and 6 describe the convex, explicit, and implicit policies. Section 7 presents the rectangle decomposition and its correctness proof. Section 8 analyzes complexity. Section 9 reports the implementation and tests. Section 10 discusses related work, and Section 11 concludes.

II. Preliminaries

A. Rectilinear polygons and orthogonal convexity

A *rectilinear* (or *orthogonal*) polygon is a simple closed polygon whose edges are axis-aligned. Its vertices alternate between horizontal and vertical edges, so it has an even number of vertices. We assume a simple polygon without holes; the algorithms extend to holes by treating each boundary component separately.

A rectilinear polygon is *orthogonally convex* (or *rectilinearly convex*) if every horizontal and every vertical line intersects it in a connected set (or in the empty set). Equivalently, it is both x-monotone and y-monotone. Orthogonal convexity is not geometric convexity. An L-shape is orthogonally convex in this sense, although its interior angle at the inner corner is 270 degrees. The implementation's predicate is exactly the conjunction

$$\text{convex}(P) \;=\; \text{xmonotone}(P) \;\wedge\; \text{ymonotone}(P).$$

Figure 1 illustrates a rectilinear polygon and the two edge orientations. Figure 2 contrasts an orthogonally convex L-shape with a U-shape that is x-monotone but not y-monotone.

B. Even-index representation

Let $v_0, \ldots, v_{n-1}$ be the vertices of a rectilinear polygon in boundary order, with n even. Rotate the list so that the edge from v_0 to v_1 is horizontal. The *even-index representation* stores

$$S_i \;=\; v_{2i}, \qquad i = 0, \ldots, k-1, \qquad k = n/2.$$

The omitted vertex between two consecutive stored points is *implicit*. With the x-first convention used throughout this paper, it is

$$c_i \;=\; (x_{i+1},\, y_i).$$

Thus the full boundary is reconstructed as

$$S_0,\; c_0,\; S_1,\; c_1,\; \ldots,\; S_{k-1},\; c_{k-1}.$$

The representation has several useful properties. It uses half as many points as the full boundary. Consecutive stored points differ in both coordinates, because the omitted vertex between them is a genuine corner. A two-point representation is a rectangle; a three-point representation is an L-shaped monotone piece; more points describe a larger orthogonally convex polygon. The stored order follows the boundary, and the full boundary is reconstructed on demand. Figure 3 shows stored and implicit vertices on an eight-vertex U-shape, and Figure 4 shows the x-first reconstruction of one stored pair.

Cut outputs may contain a degenerate consecutive pair that shares a coordinate; the reconstruction then inserts a duplicate implicit point, which yields a zero-length horizontal edge. The slab stage ignores such edges.

C. Area

The area of a polygon can be computed from the reconstructed boundary with the shoelace formula. For the compressed representation, the implementation uses the equivalent form

$$A(S) = v_1^x v_1^y + \sum_{i=2}^{k-1} v_i^x \left(v_i^y - v_{i-1}^y\right),$$
$$v_i = S_i - S_0,$$

where v_i^x and v_i^y denote the coordinates of the displacement vector v_i. For a two-point rectangle the formula reduces to $(x_1 - x_0)(y_1 - y_0)$. The sign is positive for a counterclockwise polygon and negative for a clockwise one; the orientation flag `is_anticlockwise` carries this sign through all algorithms.

Lemma 1 (compressed area). For a simple rectilinear polygon, the compressed formula equals the signed area of the reconstructed boundary.

Proof sketch. Expand the shoelace sum over the reconstructed sequence $S_0, c_0, S_1, \ldots$. Each implicit vertex contributes terms that cancel with the adjacent stored vertices, leaving the telescoping expression above. A direct expansion for the three-point L-shape and the two-point rectangle verifies the base cases; the general case follows by induction on k. □

D. Base cases

The compressed representation turns the two smallest decompositions into base cases: two stored points are a rectangle, and three stored points are an L-shaped monotone piece. These are not degenerate cases; they expand to four and six boundary vertices, respectively. Figure 5 shows the three smallest representations and their expansions.

III. The Recursive Cut Framework

A. Data structure

All cut policies operate on a circular doubly linked list whose nodes are the stored points. A cut inserts a new node into the list and appends the corresponding point

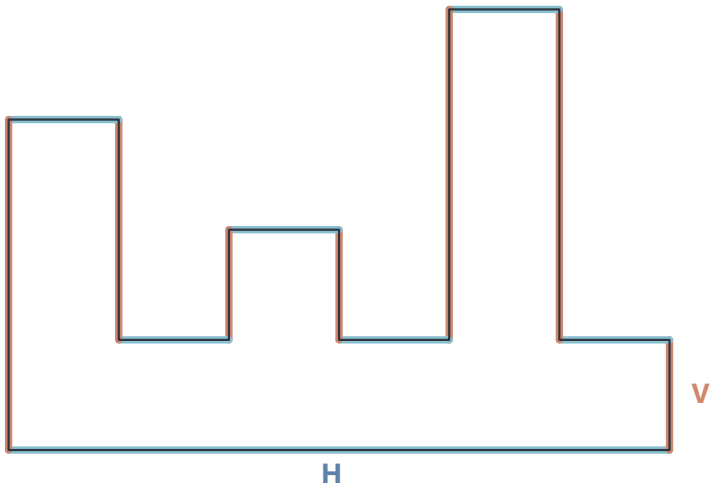


Fig. 1. A rectilinear polygon. Horizontal edges are blue; vertical edges are orange.

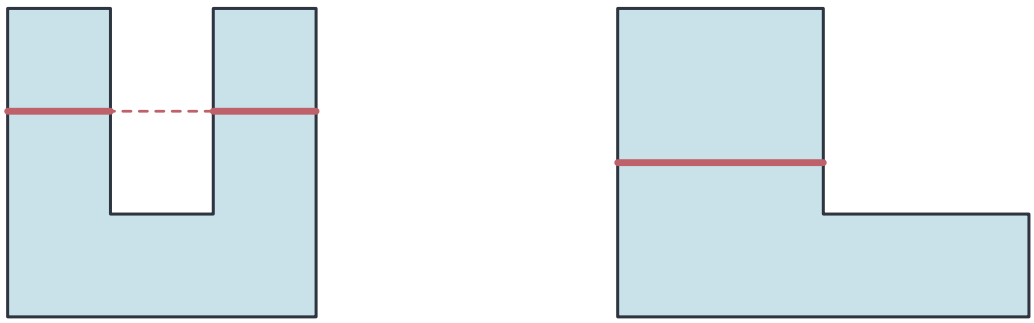


Fig. 2. Left: an L-shape is both x- and y-monotone and is therefore orthogonally convex. Right: a U-shape is x-monotone but a horizontal line meets it in two intervals, so it is not orthogonally convex.

to a point array. The recursive procedure returns lists of node indices; the wrapper converts each index list into a polygon. The linked list supports insertion and traversal in constant and linear time, respectively, and avoids the index shifting that an array-based implementation would require.

B. Skeleton

Algorithm 1 shows the shared skeleton. It is a template method: the policies differ in the base cases, in the concave-vertex predicate, and in the target of the first recursive call. The convex policy is described in Section 4; the explicit and implicit policies reuse the skeleton with different predicates.

The base cases are stored pieces, not expanded boundaries. A two-node piece is a rectangle; a three-node piece is an L-shape. The predicate `find_concave` returns the first stored vertex that violates the policy's concavity test. If no such vertex exists, the current cycle is returned as a single piece. The nearest-edge search returns a node `v_min` and a flag indicating whether the cut is vertical. The cut is inserted as a new node; the two recursive calls then process the two sides. The `first_target` hook is the only difference between the explicit and implicit recursions: the explicit policy recurses first on `v_cur`, the implicit policy on `v_min`.

C. Concave predicate

For three consecutive stored points p_0, p_1, p_2, the explicit and convex policies use the direction-reversal guard

Algorithm 1 Recursive cut skeleton

if v has exactly two nodes **then**
  return the two-node piece
end if
if the policy allows a three-node base case and v has three nodes **then**
  return the three-node piece
end if
$v_{\text{cur}} \leftarrow \textsc{findConcave}(v)$
if v_{cur} is none **then**
  return the whole cycle as one piece
end if
$(v_{\min}, \textit{vertical}) \leftarrow \textsc{findMinDist}(v_{\text{cur}})$
if a degenerate two-node split applies **then**
  return $\textsc{split}(v_{\text{cur}}, v_{\min})$
end if
$p_{\text{new}} \leftarrow \textsc{insertCut}(v_{\text{cur}}, v_{\min}, \textit{vertical})$
$u \leftarrow \textsc{firstTarget}(v_{\text{cur}}, v_{\min})$
return $\textsc{recur}(u) + \textsc{recur}(\textit{newNode})$

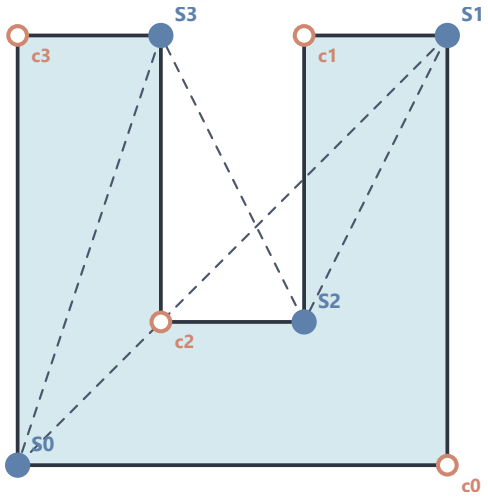


Fig. 3. Stored (filled blue) and implicit (hollow orange) vertices of an eight-vertex U-shape. Dashed segments connect consecutive stored points; they are not polygon edges. The compressed representation has four stored points.

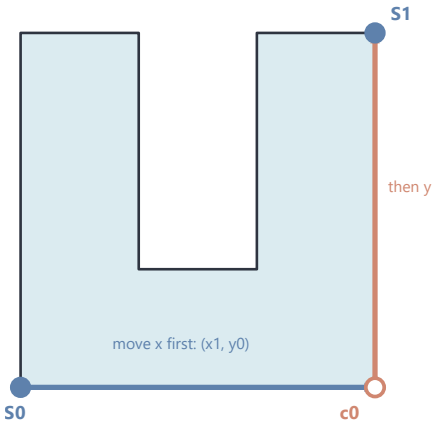


Fig. 4. Reconstruction of the edge from S_0 to S_1: move horizontally to $c_0 = (x_1, y_0)$, then vertically to S_1. The y-first alternative reconstructs a different polygon.

$$R(p_1) = \big[(p_1^x - p_0^x)(p_2^x - p_1^x) < 0\big] \\ \vee \big[(p_1^y - p_0^y)(p_2^y - p_1^y) < 0\big].$$

and the signed-area term

$$a(p_1) \;=\; (p_1^y - p_0^y)(p_2^x - p_1^x).$$

The vertex is treated as concave when $R(p_1)$ holds and $\text{cmp}(a(p_1))$ is true, where

$$\text{cmp}(a) \;=\; \begin{cases} a > 0, & \text{if the polygon is counter-clockwise,} \\ a < 0, & \text{if the polygon is clockwise.} \end{cases}$$

This is the compressed-list form of the reflex test. It should not be read as a test for every reflex vertex of the reconstructed boundary. The inner corner of an L-shape is a reflex vertex of the full boundary, yet the compressed predicate does not cut it, because the L-shape is already orthogonally convex. The predicate detects a stored corner at which the compressed chain fails the monotonicity condition; its role is to drive the partition into orthogonally convex pieces, not to enumerate all 270-degree corners.

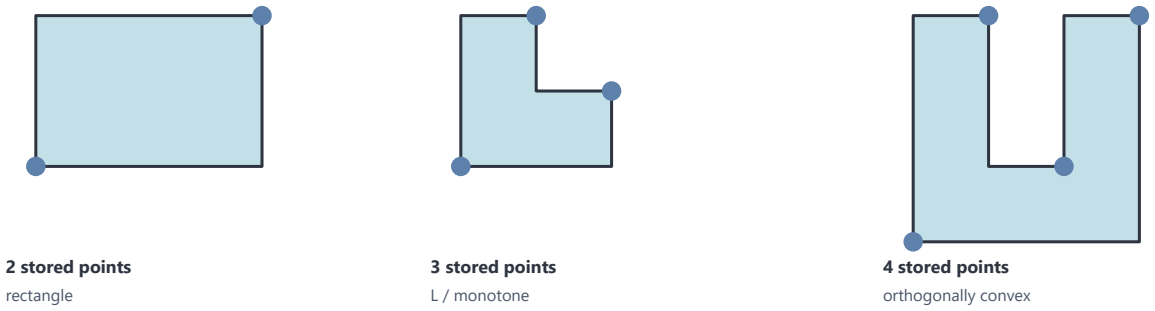


Fig. 5. Base cases of the compressed representation: two stored points (rectangle), three stored points (L-shape), and four stored points (orthogonally convex polygon).

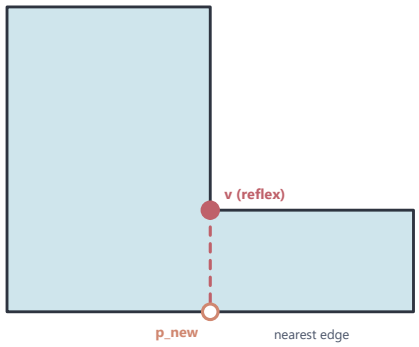


Fig. 6. A cut from a stored reflex vertex v to the nearest edge. The new point p_{new} is inserted into the circular list; the two resulting pieces are processed recursively.

D. Nearest edge and cut insertion

Given a concave vertex v, the algorithm projects v vertically and horizontally onto the boundary and selects the nearest edge. For the explicit and convex policies the projection point is v itself; for the implicit policy it is the implicit corner $c = (x_{\text{next}}, y_{\text{cur}})$. The search examines each edge in the circular list and keeps the minimum distance. If the nearest edge is vertical, the new cut point is

$$p_{\text{new}} = (x_{\min}, y_{\text{cur}});$$

if it is horizontal,

$$p_{\text{new}} = (x_{\text{cur}}, y_{\min}).$$

The new node is spliced between the appropriate neighbours. Figure 6 shows a cut from a stored reflex vertex to the nearest edge.

E. Degenerate cuts

A cut point can coincide with an existing vertex. When the nearest target is adjacent to the current vertex in the circular list and the cut point duplicates that target, inserting a new node would create a zero-length edge. The implementation detects this situation and splits off the existing two-node piece instead of inserting a duplicate. The twelve-point fixture exercises this case: the first cut at the stored vertex $(3, 2)$ targets $(1, 2)$, which is adjacent in the list, so the two-node piece $(2, 3), (1, 2)$ is split off as a unit. The remaining two pieces are the six-point piece of area 10 and the five-point piece of area 9. Without the degenerate split, the same run returns a single nineteen- area piece and a one-area piece, and the three-piece partition is lost.

IV. Convex Decomposition

A. Algorithm

The convex policy cuts until no stored triple satisfies the predicate of Section 3.3. Its base cases are the two-node rectangle and the three-node monotone piece. The implementation uses a standalone recursion with the same steps as Algorithm 1; the shared skeleton is retained for the explicit and implicit policies.

B. Correctness

Lemma 2 (cut partition). A cut from a vertex to an interior edge splits a simple rectilinear polygon into two simple rectilinear polygons whose interiors are disjoint and whose areas sum to the area of the original polygon.

Proof sketch. The cut segment lies in the interior except at its endpoints, which lie on the boundary. Adding the segment to the boundary produces two closed rectilinear cycles. The cycles are simple because the segment does not cross the boundary, and the area identity follows from the additivity of the shoelace sum. □

Theorem 1 (convex decomposition). The convex recursion terminates and returns a partition of the input polygon into pieces that satisfy the orthogonal-convexity

predicate of Section 2.1. The area of the pieces sums to the area of the input.

Proof sketch. Each cut strictly reduces the number of vertices on each side of the recursion, so the recursion terminates. Lemma 2 gives the partition and the area identity. The stopping condition is the absence of a stored triple that satisfies the compressed concavity predicate; the implementation verifies on every returned piece that the x- and y-monotone tests hold. This is the invariant used by the rectangle stage in Section 7. A fully formal proof that the compressed predicate characterizes orthogonal convexity is beyond the scope of this paper; the implementation's test suite asserts the invariant on all fixtures. □

C. Example

The twelve-point fixture has stored points

$$(-1,1),\ (0,0),\ (1,-1),\ (3,0),\ (4,1),\ (6,2),\\ (5,5),\ (4,4),\ (3,2),\ (2,3),\ (1,2),\ (0,3).$$

and area 20. The convex policy returns three pieces, listed in Table 1 and shown in Figure 7. The two-node piece is a rectangle; the other two are orthogonally convex but not rectangles. This distinction is the reason a convex decomposition cannot be used as a rectangle decomposition without a second stage.

V. Explicit Decomposition

The explicit policy uses the same recursive skeleton and the same concavity predicate as the convex policy, but it does not use the three-node base case. Consequently, a three-node piece is not returned automatically; the recursion continues if the predicate fires. In practice the policy is an alternative convex decomposition. On the U-shape of Figure 8, both policies return the same two pieces, of areas 4 and 3. The explicit policy is not a rectangularizer: a piece that is already orthogonally convex has no vertex satisfying the predicate, so the recursion returns it whole.

This is the key limitation that the old rectangle composition overlooked. Applying the explicit policy to the five-point piece P_2 of area 9 returns P_2 unchanged, because P_2 is orthogonally convex. The explicit stage can therefore remove concavities that the implicit stage leaves behind, but it cannot turn a convex piece into rectangles.

Table 2 summarizes the three policies.

VI. Implicit Decomposition

A. Implicit corners

The implicit policy operates directly on stored pairs. For a stored point $p_1 = S_i$ and its successor $p_2 = S_{i+1}$, the corner under test is the implicit vertex

$$c_i \ = \ (p_2^x,\, p_1^y).$$

The policy uses the sign test

$$a_i \ = \ -(p_2^y - p_1^y)(p_2^x - p_1^x)$$

and cuts at p_1 when $\mathrm{cmp}(a_i)$ holds. There is no direction-reversal guard: the implicit vertex is a genuine 90-degree corner by construction, so the sign test alone decides whether it is reflex in the orientation of the polygon. The nearest-edge search uses c_i as the projection point and starts one node after p_2. The cut is inserted at

$$p_{\text{new}} \ = \ (x_{\min},\, c_i^y) \quad \text{or} \quad p_{\text{new}} \ = \ (c_i^x,\, y_{\min}),$$

and the recursion processes the nearest-edge side first, then the new node.

B. Example

The L-shape stored as $(0,0), (2,1), (1,2)$ has one implicit corner, $(1,1)$. The relevant stored pair is $(2,1), (1,2)$. The test value is

$$-(2-1)(1-2) = +1,$$

which is positive, so the counter-clockwise orientation selects the implicit corner $(1,1)$. The cut from $(1,1)$ horizontally to the nearest edge splits the L-shape into the rectangles $(0,0), (2,1)$ and $(0,1), (1,2)$, of areas 2 and 1. The result is shown in Figure 9.

C. The blind spot

Proposition 1 (implicit blind spot). The implicit policy only cuts at implicit corners. A reflex corner that is a stored vertex is never tested, so a polygon whose relevant reflex corners are stored can be returned as a piece that is not a rectangle.

Proof. The predicate of Section 6.1 is evaluated on each pair (S_i, S_{i+1}) and returns S_i as the cut candidate. It never evaluates a stored vertex against its predecessor and successor in the full boundary. If every reflex corner of the current piece is a stored vertex, every test value is non-positive for a counter-clockwise piece, and the recursion returns the piece unchanged. □

The five-point piece P_2 of area 9 is a concrete witness. Its stored points are

$$(3,0), (4,1), (6,2), (5,5), (4,4).$$

The implicit cut fires once, at $(6,2)$, and returns the four-point piece

$$(4,1), (5,5), (4,4), (3,0)$$

TABLE I
CONVEX DECOMPOSITION OF THE TWELVE-POINT FIXTURE. THE AREAS SUM TO 20.

Piece	Stored points	Area
P_0	$(2,3),(1,2)$	1
P_1	$(3,2),(0,3),(-1,1),(0,0),(1,-1),(3,0)$	10
P_2	$(3,0),(4,1),(6,2),(5,5),(4,4)$	9

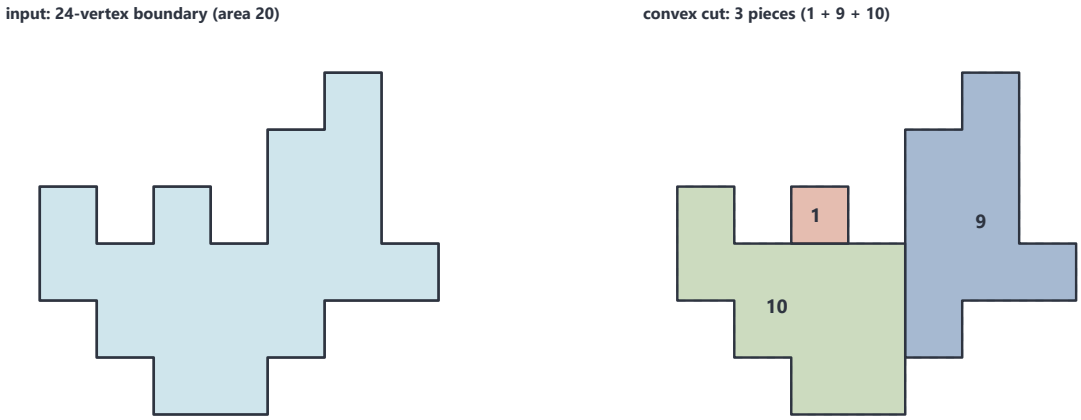


Fig. 7. Convex decomposition of the twelve-point fixture: three orthogonally convex pieces of areas 1, 9, and 10.

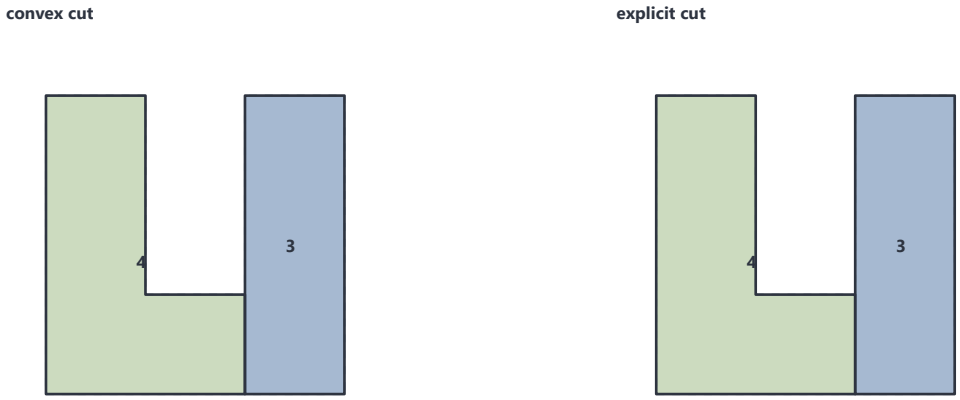


Fig. 8. Convex and explicit cuts on a U-shape. Both return the same two pieces here; the explicit policy differs in its stopping rule, not in its geometric criterion.

TABLE II
COMPARISON OF THE THREE CUT POLICIES.

Policy	Base cases	Predicate	Typical output
Convex	2 and 3 nodes	compressed reflex test	orthogonally convex pieces
Explicit	2 nodes	compressed reflex test	orthogonally convex pieces
Implicit	2 nodes	implicit-corner test	pieces cut at implicit corners

of area 8 together with the rectangle $(5,1),(6,2)$ of area 1. The four-point piece expands to an octagon; it is orthogonally convex but not a rectangle. On the residual four-point piece the test values are $-4,-1,-4,-1$, all negative, so the implicit policy stops. Figure 10 shows the result.

The raw twelve-point fixture exhibits a second symptom. Running the implicit policy on the fixture itself produces six pieces, including the zero-width piece $(0,2),(0,3)$ of area 0. The piece has the same x-coordinate at both ends, so it is not a valid two-point rectangle. Area preservation does not detect it: the areas of the six pieces are $2,1,14,0,1,2$, which sum to 20. This is the failure mode that motivates

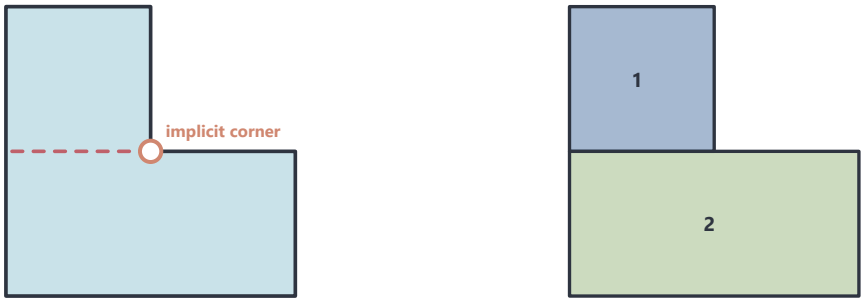


Fig. 9. Implicit cut of an L-shape. The implicit corner $(1, 1)$ is reflex; a horizontal cut yields two rectangles of areas 2 and 1.



Fig. 10. The implicit blind spot. On the area-9 convex piece, the implicit cut leaves an 8-area four-point piece (an octagon after expansion) and a 1-area rectangle. The stored reflex corners are never tested.

the corrected rectangle pipeline.

VII. Rectangle Decomposition

A. Why implicit-then-explicit fails

The natural composition is

$$\text{rect}(P) \;=\; \bigcup_{Q \in \text{implicit}(P)} \text{explicit}(Q).$$

On the twelve-point fixture it returns six pieces, of areas $2, 1, 14, 0, 1, 2$. Two defects are visible. First, the zero-width piece $(0, 2), (0, 3)$ is not a rectangle. Second, the five-point piece of area 14 is orthogonally convex but not a rectangle; the explicit stage returns it unchanged for the reason given in Section 5. The composition is therefore not a rectangle decomposition, even though its area sum is correct. Area preservation is a necessary but not sufficient invariant.

Figure 11 contrasts the old composition with the corrected pipeline. The old output contains a zero-width piece and a five-point non-rectangle; the new output contains seven proper rectangles and the same total area.

B. Convex plus slabs

The corrected algorithm has two stages:

1. Compute the convex decomposition $C = \text{convex}(P)$.
2. For each $Q \in C$, rectangularize Q by vertical slabs.

The slab stage is simple. Let $x_1 < \cdots < x_m$ be the distinct x-coordinates of the stored points of Q. For each slab $[x_j, x_{j+1}]$, collect the y-coordinates of the horizontal edges that span the slab, sort them, and pair consecutive values. For an x-monotone piece there are exactly two values per slab, and the slab contributes the rectangle

$$[x_j, x_{j+1}] \;\times\; [y_{\text{lo}}, y_{\text{hi}}].$$

The rectangle is stored as two opposite corners. For a counter-clockwise piece the stored order

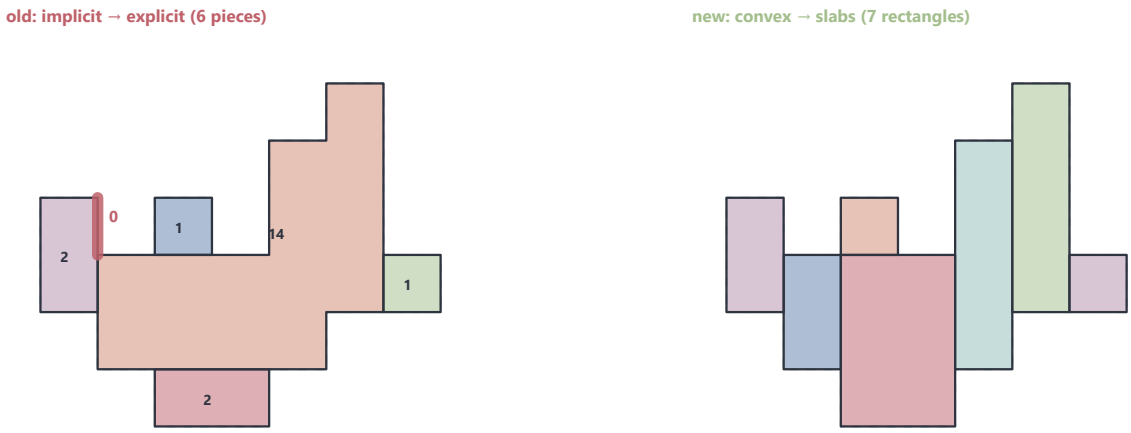


Fig. 11. Old composition (implicit then explicit) versus the corrected pipeline (convex then slabs). The old output contains a zero-width piece and a five-point non-rectangle; the new output contains seven rectangles with the same total area.

is $(x_j, y_{\text{lo}}), (x_{j+1}, y_{\text{hi}})$; for a clockwise piece it is $(x_j, y_{\text{hi}}), (x_{j+1}, y_{\text{lo}})$, so that the signed area keeps the sign of the input. Algorithm 2 states the procedure.

The pairing assumes that the number of crossings is even. For an x-monotone piece it is exactly two; the even-odd pairing also gives a valid decomposition for a non-monotone piece, but the convex stage is what guarantees the simple one-rectangle-per-slab case.

C. Correctness

Lemma 3 (slab rectangle). Let Q be an x-monotone rectilinear polygon and let $[x_j, x_{j+1}]$ be a slab between consecutive distinct vertex x-coordinates. Then $Q \cap ([x_j, x_{j+1}] \times \mathbb{R})$ is a rectangle.

Proof. For every $x \in (x_j, x_{j+1})$, x-monotonicity implies that the intersection of Q with the vertical line at x is a single interval $[y_{\text{lo}}(x), y_{\text{hi}}(x)]$. No vertex x-coordinate lies strictly inside the slab, so the boundary edges that cross the slab are horizontal and their y-coordinates are constant on the slab. Hence $y_{\text{lo}}(x) = y_{\text{lo}}$ and $y_{\text{hi}}(x) = y_{\text{hi}}$ are independent of x, and the intersection is the rectangle $[x_j, x_{j+1}] \times [y_{\text{lo}}, y_{\text{hi}}]$. □

Theorem 2 (rectangle decomposition). Let C be a partition of a rectilinear polygon P into x-monotone pieces. Then the slab stage applied to every piece of C returns a partition of P into rectangles, and the areas of the rectangles sum to the area of P.

Proof. Lemma 3 shows that each slab of each piece contributes one rectangle, and the slabs of a piece partition it. The pieces of C partition P, so the rectangles partition P. Areas are additive over a partition, so the area sum is preserved. The two-point storage is a valid compressed rectangle by construction. □

The premise of Theorem 2 is exactly the invariant of the convex stage (Theorem 1): every returned piece satisfies the x- and y-monotone tests. The implementation checks the invariant on every test fixture. The theorem therefore reduces the rectangle problem to the convex decomposition and a simple slab pass.

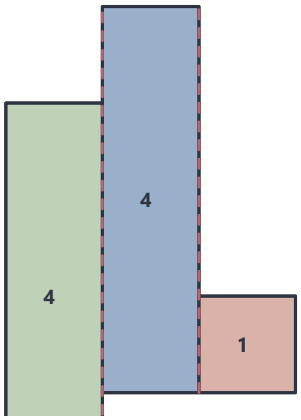


Fig. 12. Slab decomposition of the area-9 convex piece: three vertical slabs yield three rectangles of areas 4, 4, and 1.

D. Example

Figure 12 shows the slab decomposition of the area-9 piece P_2. The distinct x-coordinates are $3, 4, 5, 6$, giving three slabs. The slab $[3, 4]$ contributes the rectangle $(3, 0), (4, 4)$ of area 4; the slab $[4, 5]$ contributes $(4, 1), (5, 5)$ of area 4; the slab $[5, 6]$ contributes $(5, 1), (6, 2)$ of area 1. The sum is 9.

Applying the pipeline to the full fixture yields seven rectangles, listed in Table 3 and shown in Figure 13. Each piece is stored as two opposite corners with distinct x- and y-coordinates, and the areas sum to 20.

Algorithm 2 Rectangularize an x-monotone piece

```
xs ← sorted distinct x-coordinates of Q
for j = 1 to m − 1 do
    ys ← ∅
    for each stored edge (p, q) of Q do
        if min(p^x, q^x) ≤ xs_j and xs_{j+1} ≤ max(p^x, q^x) then
            append p^y to ys
        end if
    end for
    sort ys
    for i = 0, 2, 4, ... do
        if ys_i ≠ ys_{i+1} then
            emit rectangle (xs_j, ys_i) to (xs_{j+1}, ys_{i+1})
        end if
    end for
end for
```

TABLE III
Rectangle decomposition of the twelve-point fixture. The areas sum to 20.

Piece	Stored corners	Area
R1	$(1,-1),(3,2)$	6
R2	$(0,0),(1,2)$	2
R3	$(1,2),(2,3)$	1
R4	$(-1,1),(0,3)$	2
R5	$(3,0),(4,4)$	4
R6	$(4,1),(5,5)$	4
R7	$(5,1),(6,2)$	1

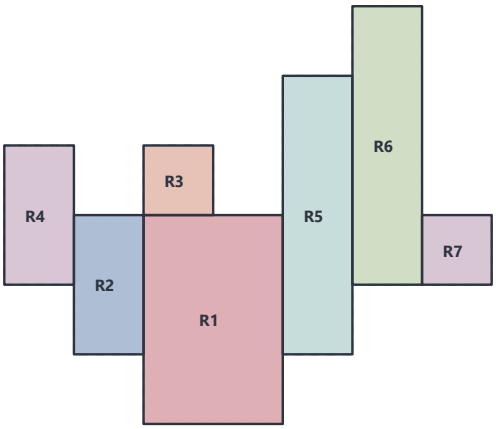


Fig. 13. Rectangle decomposition of the twelve-point fixture: seven rectangles of areas 6, 2, 1, 2, 4, 4, and 1.

E. Remarks

The algorithm is not a minimum-cardinality rectangle partition. The slab decomposition depends on the vertex x-coordinates and can produce more rectangles than a carefully chosen cut sequence. Minimum rectangle partition is a separate optimization problem with a substantial literature (Section 10); the goal here is a simple, deterministic, and provably correct pipeline that is easy to implement on the compressed representation. The slab stage is also embarrassingly local: each piece is processed independently and no global data structure is required.

VIII. Complexity

Let n be the number of stored points of the input polygon.

Cut stage. The nearest-edge search scans $O(n)$ edges. Each cut increases the number of pieces by one and the recursion creates at most $O(n)$ pieces, so the cut stage takes $O(n^2)$ time. The linked list supports insertion in $O(1)$ and the point array is appended in amortized $O(1)$. The recursion uses $O(n)$ stack space in the worst case and $O(n)$ storage for the pieces.

Slab stage. For a piece with k stored points, sorting the distinct x-coordinates takes $O(k \log k)$; each of the $O(k)$ slabs scans $O(k)$ edges, so the slab stage takes $O(k^2)$ time. Summed over the pieces of the convex decomposition, the total is $O(n^2)$ in the worst case. The output has $O(n)$ rectangles.

Overall. The pipeline runs in $O(n^2)$ time and $O(n)$ extra space. On the twelve-point fixture the convex stage returns three pieces after two cuts, and the slab stage returns seven rectangles.

IX. Implementation and Verification

The algorithms are implemented in C++20 as part of the Recti physical-design library. The public entry point is

```
template <typename T>
auto rpolygon_cut_rectangle(
    std::span<const Point<T>> pointset,
    bool is_anticlockwise)
    -> std::vector<std::vector<Point<T>>>;
```

The input is the even-index representation of a rectilinear polygon and the orientation flag; the output is a vector of compressed rectangles. The convex, explicit, and implicit policies remain available as separate functions.

The implementation contains three details that matter in practice. First, the convex cut has a degenerate two-node split (Section 3.5) that reuses an existing two-node piece when the cut point duplicates an adjacent vertex; without it the twelve-point fixture is not split into three pieces. Second, the slab stage skips zero-height pairs, so it never emits a zero-area rectangle. Third, the rectangle output is asserted to be a two-point piece with distinct x- and y-coordinates and positive area; this is the invariant that area-only tests miss.

The test fixture is the twelve-point stored list of Section 4.3. The convex cut is required to return three pieces with areas $1, 9, 10$; the rectangle cut is required to return seven rectangles with sorted areas $1, 1, 2, 2, 4, 4, 6$; every rectangle must have two stored points, distinct coordinates, and positive area; and the total area must be 20. The fixture also checks that the invalid point $(2, 2)$ does not appear in any output piece. Table 4 lists the expectations.

The full suite contains 138 test cases and 837 assertions and passes under `xmake`. The tests include random rectilinear polygons, signed-area preservation, convexity of every convex piece, and the fixture above. The old composition is also covered by a regression test: it must not be used as the rectangle algorithm, and the new output must satisfy the two-point rectangle invariant.

X. Related Work

Rectilinear polygon decomposition is a classical topic in computational geometry and in very-large-scale integration (VLSI) layout. The minimum-number rectangle partition problem was studied by Ohtsuki [1], who gave an $O(n^{5/2})$ algorithm for polygons with holes. Lipski [2] developed a Manhattan-path approach that yields an $O(n^{3/2} \log n)$ minimum dissection, and Nahar and Sahni [3] gave a fast horizontal-cut decomposition whose running time depends on a structural parameter of the polygon. Soltan and Gorpinevich [4] gave an $O(n^{3/2} \log n)$ algorithm for arbitrary holes. For convex decomposition, Keil [5] gave a polynomial dynamic-programming algorithm for simple polygons, while Lingas [6] showed that minimum convex partition becomes NP-hard in the presence of holes. Keil's survey [7] and the textbook of de Berg et al. [8] provide the standard background.

The present work differs from the optimization literature in two ways. First, it focuses on a compressed even-index representation and on the implementation of the cut primitives on that representation. Second, it isolates a correctness issue in a natural composition of two cut policies: the implicit policy is not a rectangularizer, and the explicit policy cannot finish the job on a convex piece. The corrected pipeline is deliberately simple; it makes no optimality claim and is intended as a reliable preprocessing step. The slab stage is a standard vertical decomposition restricted to x-monotone pieces; its role here is to provide a provably correct second stage for the compressed cut framework.

XI. Conclusion

We studied the decomposition of rectilinear polygons into convex pieces and rectangles on an even-index representation. The convex, explicit, and implicit policies share a recursive cut skeleton but differ in their predicates and base cases. The implicit policy only sees implicit corners; a reflex corner that is a stored vertex is invisible to it. The explicit policy stops on an orthogonally convex piece. Consequently, implicit-then-explicit is not a rectangle decomposition, even though it preserves area. The twelve-point fixture makes the failure concrete: six pieces, including a zero-width piece and a five-point piece of area 14, with a correct area sum of 20.

The corrected pipeline first computes a convex decomposition and then rectangularizes each x-monotone piece with vertical slabs. We proved that the slab stage returns a partition into rectangles for x-monotone pieces and analyzed the $O(n^2)$ time complexity. The C++ implementation returns seven rectangles of areas $6, 2, 1, 2, 4, 4, 1$ on the fixture and passes a 138-case test suite. The main lesson is that tests for a decomposition must assert the output class, not only the area: a zero-width piece and a five-point non-rectangle can preserve the area while violating the specification.

Future work includes minimum-cardinality rectangle partitioning on the compressed representation, extensions to polygons with holes, and a formal proof that the compressed concavity predicate characterizes orthogonal convexity.

References

[1] T. Ohtsuki, "Minimum dissection of rectilinear regions," in *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, Rome, Italy, 1982, pp. 1210-1213.

[2] W. Lipski, Jr., "An O(n log n) Manhattan path algorithm," *Information Processing Letters*, vol. 19, no. 2, pp. 99-102, 1984. DOI: 10.1016/0020-0190(84)90105-4.

[3] S. Nahar and S. Sahni, "Fast algorithm for polygon decomposition," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 7, no. 4, pp. 473-483, 1988. DOI: 10.1109/43.3182.

[4] V. G. Soltan and A. Gorpinevich, "Minimum dissection of a rectilinear polygon with arbitrary holes into rectangles," *Discrete and Computational Geometry*, vol. 9, no. 1, pp. 57-79, 1993. DOI: 10.1007/BF02189307.

TABLE IV
TEST EXPECTATIONS FOR THE TWELVE-POINT FIXTURE.

Property	Convex cut	Rectangle cut
Number of pieces	3	7
Areas	1, 9, 10	1, 1, 2, 2, 4, 4, 6
Total area	20	20
Piece form	convex	two-point rectangle
Invalid point $(2, 2)$	absent	absent

[5] J. M. Keil, "Decomposing a polygon into simpler components," *SIAM Journal on Computing*, vol. 14, no. 4, pp. 799-817, 1985. DOI: 10.1137/0214056.

[6] A. Lingas, "The power of non-rectilinear holes," in *Automata, Languages and Programming (ICALP)*, Lecture Notes in Computer Science, vol. 140. Berlin: Springer, 1982, pp. 369-383. DOI: 10.1007/BFb0012784.

[7] J. M. Keil, "Polygon decomposition," in *Handbook of Computational Geometry*, J.-R. Sack and J. Urrutia, Eds. Amsterdam: Elsevier, 2000, pp. 491-518. DOI: 10.1016/B978-044482537-7/50012-7.

[8] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed. Berlin: Springer, 2008. DOI: 10.1007/978-3-540-77974-2.

Appendix A: Fixture Data

The twelve-point fixture is stored as

```
(-1,1) (0,0) (1,-1) (3,0)
(4,1) (6,2) (5,5) (4,4)
(3,2) (2,3) (1,2) (0,3)
```

Its full boundary is

```
(-1,1) (0,1) (0,0) (1,0)
(1,-1) (3,-1) (3,0) (4,0)
(4,1) (6,1) (6,2) (5,2)
(5,5) (4,5) (4,4) (3,4)
(3,2) (2,2) (2,3) (1,3)
(1,2) (0,2) (0,3) (-1,3)
```